

TASK 10

Containerizing Flask Deep Learning API

Deployment Report

Student Name	Aditya Laxman bahira
Course / Program	MSc Data Science / Deep Learning
Technology	Flask, PyTorch, Docker

1. Objective

The objective of this practical is to package a Flask-based deep learning REST API into a Docker container and deploy it locally. The implementation covers creation of a Dockerfile, Docker image building, container execution, API verification, model inference, container lifecycle operations, logging, and basic image-size optimization.

2. Application Overview

The existing application is a Flask REST API for a health-risk deep learning model. The API loads a trained PyTorch model during application startup and exposes endpoints for service information, health/readiness checking, prediction, and interactive API documentation.

Implemented API endpoints:

- GET / — API landing page and service metadata.
- GET /health — server and model health/readiness status.
- POST /predict — deep learning inference using submitted feature values.
- GET /docs — interactive HTML API documentation.

3. Project Structure

The project used for containerization contains the Flask application, model loader, dependencies, trained model files, and Docker configuration.

```
Task_10-Containerizing_Flask_Deep_Learning_API/  
|  
├── saved_models/  
├── app.py  
├── model_loader.py  
├── test_client.py  
├── requirements.txt  
├── Dockerfile  
├── .dockerignore  
├── health_activity_data.xlsx  
└── other report/support files
```

4. Dockerfile

The following Dockerfile was used to package the Flask API:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

5. Docker Build Configuration

A `.dockerignore` file was used to prevent unnecessary development and report files from being copied into the Docker build context. The trained `saved_models` directory was retained because it is required by the application for inference.

```
__pycache__
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
*.docx
*.xlsx
build_task10_doc.py
```

6. Docker Image Creation

The Docker image was built using the following command:

```
docker build -t health-risk-flask-api:1.0 .
```

The resulting image was verified using `docker images`. The image packages the Python runtime, Flask application, required Python dependencies, model loader, and trained model files.

7. Container Deployment

The Flask API was deployed as a detached Docker container using:

```
docker run -d --name health-risk-api -p 5000:5000 health-risk-flask-api:1.0
```

The port mapping connects host port 5000 to container port 5000. The Flask application is configured to listen on `0.0.0.0:5000`, allowing requests from the host system to reach the container.

8. Container Verification

docker ps – Verified that the `health-risk-api` container was running.

docker ps -a – Displayed the complete container list, including stopped containers.

docker logs health-risk-api – Displayed Flask startup and application/model initialization logs.

docker stats health-risk-api – Displayed live container resource usage.

9. API Verification

9.1 Root Endpoint

The root endpoint was accessed through:

`http://localhost:5000/`

The endpoint returned the API service metadata and listed the available API endpoints.

9.2 Health Endpoint

The health endpoint was tested through:

`http://localhost:5000/health`

A successful health response confirms that the inference engine is initialized and the model is available.

9.3 Interactive Documentation

The documentation page was verified through:

`http://localhost:5000/docs`

The page provides an overview of the API endpoints and a sample prediction request/response.

9.4 Prediction Endpoint

The POST `/predict` endpoint was tested using a JSON feature vector. The endpoint validates the input, performs PyTorch model inference, and returns prediction information, confidence scores, latency, model name, and a timestamp.

10. Container Lifecycle Operations

```
docker stop health-risk-api
```

```
docker start health-risk-api
```

```
docker restart health-risk-api
```

```
docker rm -f health-risk-api
```

These commands were used to demonstrate basic Docker container lifecycle management.

11. Optimization Practices

- Used the lightweight `python:3.11-slim` base image instead of a full Python image.
- Used `pip --no-cache-dir` to avoid retaining the package download cache inside the image.
- Used `.dockerignore` to exclude unnecessary files from the build context.
- Copied `requirements.txt` before application files so the dependency layer can be reused when application code changes.
- Kept the trained model files in the image because they are required for local inference.

Image inspection commands used:

```
docker images health-risk-flask-api
```

```
docker history health-risk-flask-api:1.0
```

```
docker inspect health-risk-flask-api:1.0
```

12. Screenshots and Evidence

Screenshots captured during implementation should be inserted in this section. Each screenshot should be placed immediately below its corresponding caption.

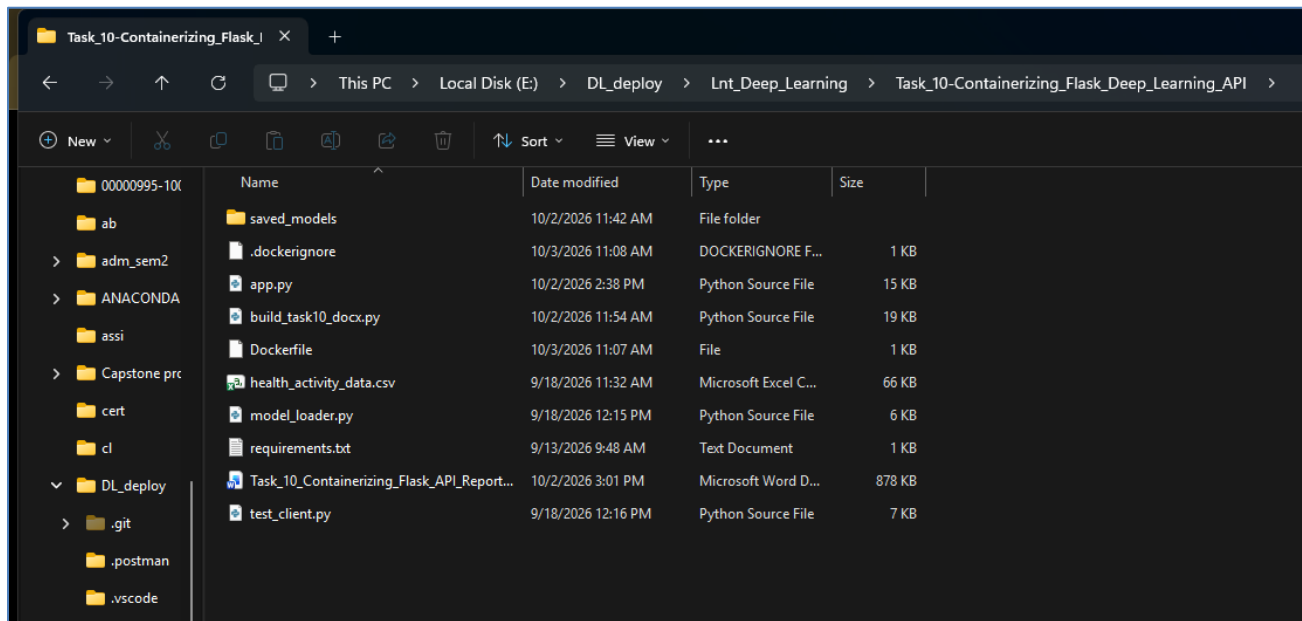


Figure 1-Project directory and Dockerfile

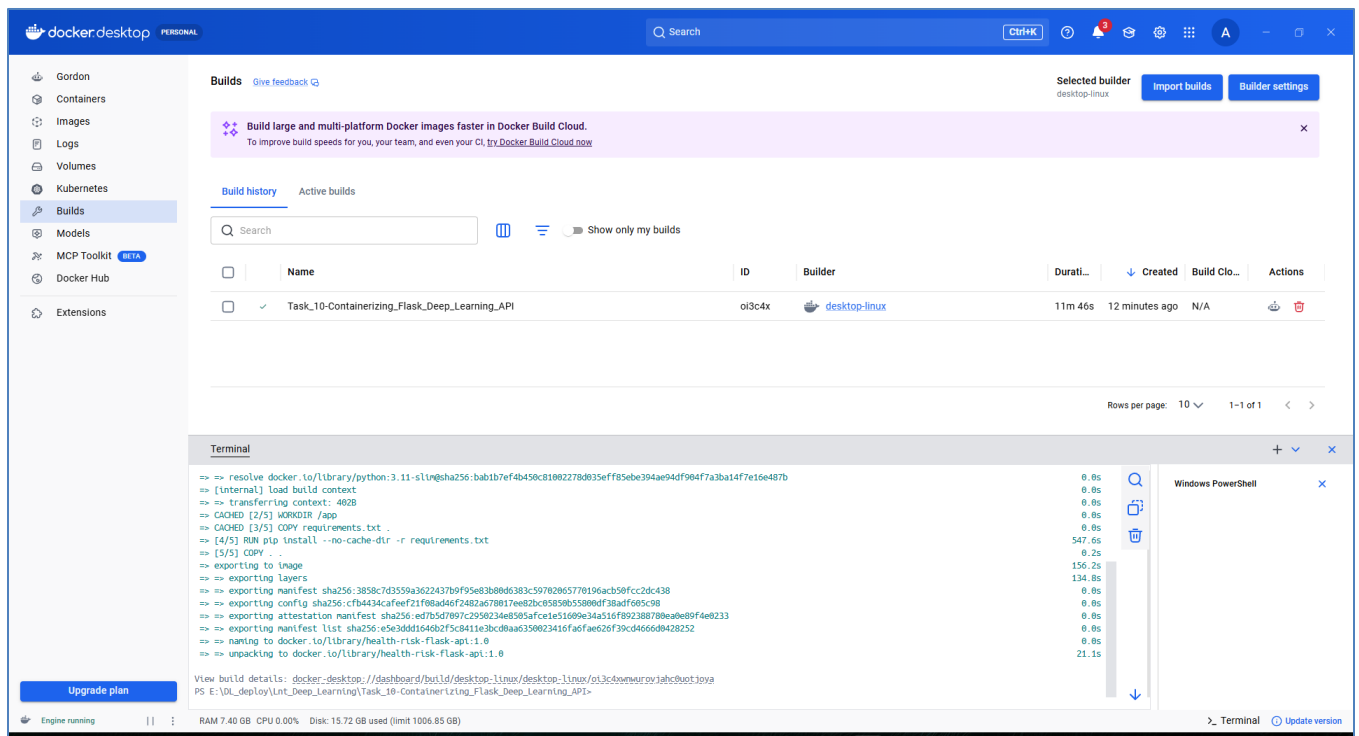


Figure 2-Docker image build output

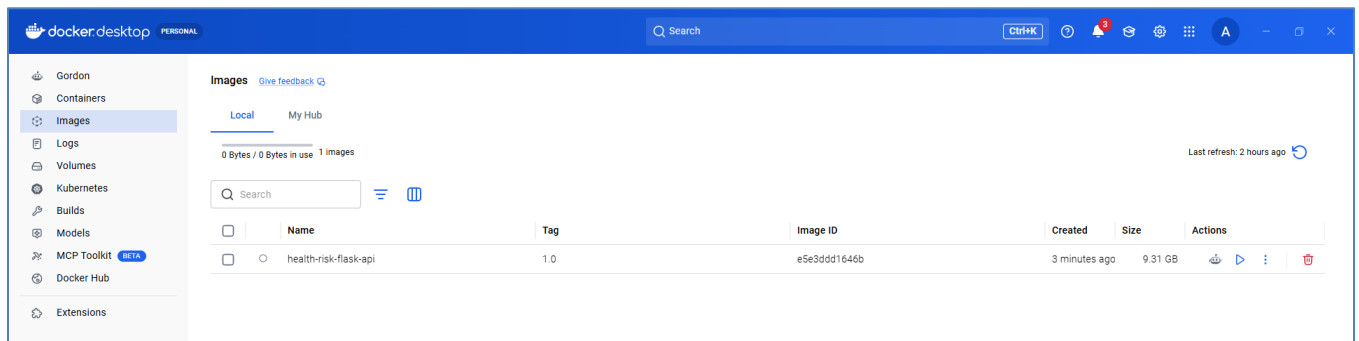


Figure 3-Docker image listed in Docker Desktop / docker images

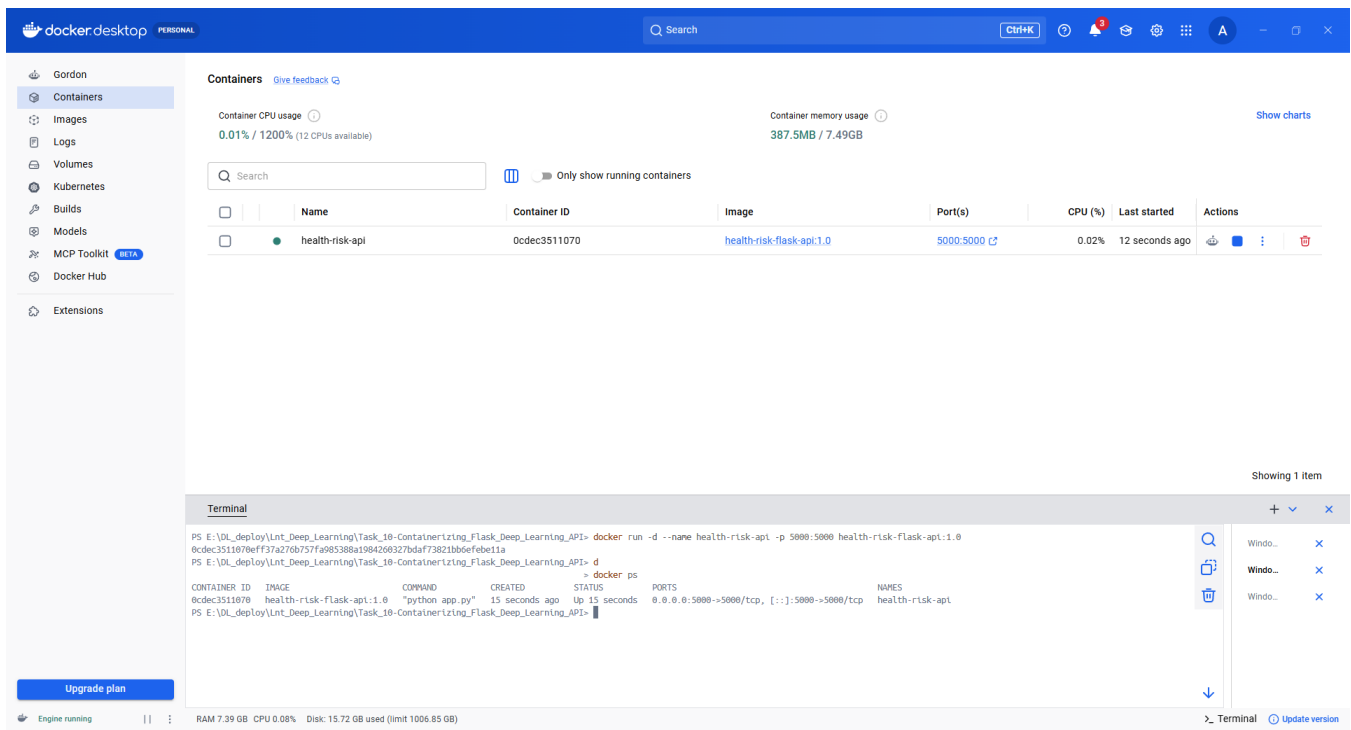


Figure 4-Running container and port mapping

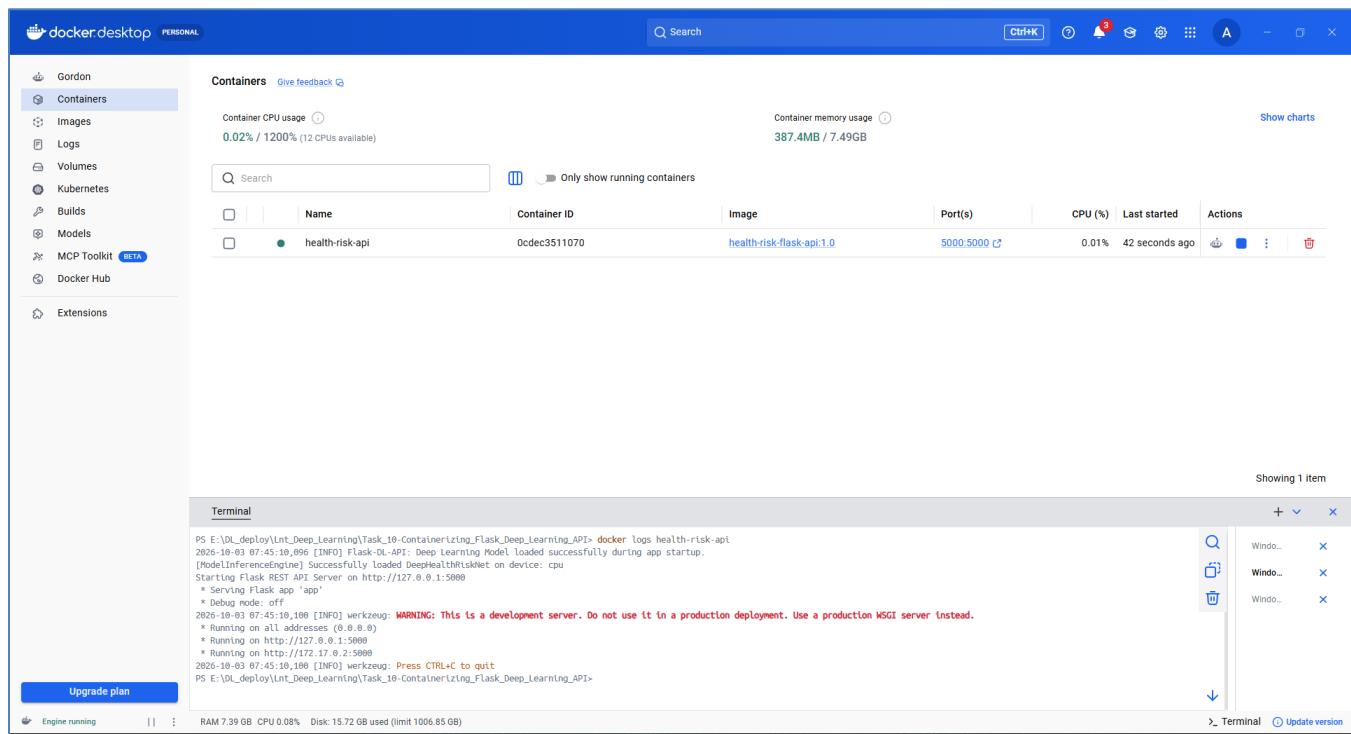
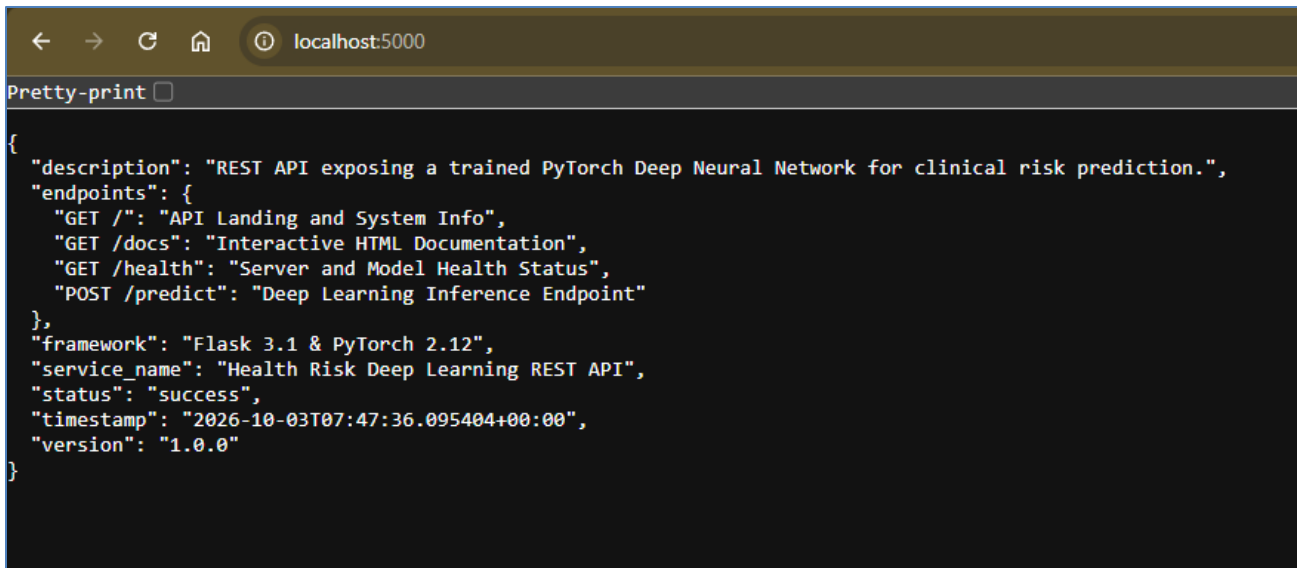
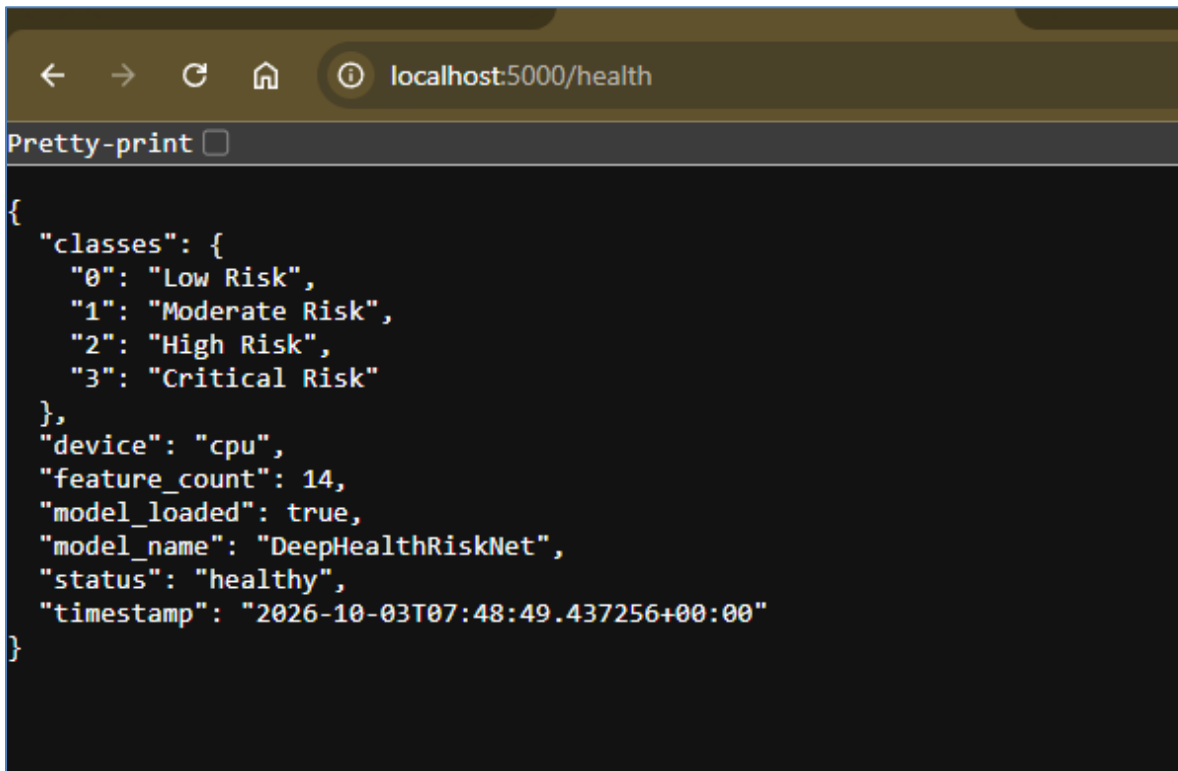


Figure 5-Docker container logs



```
{
  "description": "REST API exposing a trained PyTorch Deep Neural Network for clinical risk prediction.",
  "endpoints": {
    "GET /": "API Landing and System Info",
    "GET /docs": "Interactive HTML Documentation",
    "GET /health": "Server and Model Health Status",
    "POST /predict": "Deep Learning Inference Endpoint"
  },
  "framework": "Flask 3.1 & PyTorch 2.12",
  "service_name": "Health Risk Deep Learning REST API",
  "status": "success",
  "timestamp": "2026-10-03T07:47:36.095404+00:00",
  "version": "1.0.0"
}
```

Figure 6-Root API endpoint response



```
{
  "classes": {
    "0": "Low Risk",
    "1": "Moderate Risk",
    "2": "High Risk",
    "3": "Critical Risk"
  },
  "device": "cpu",
  "feature_count": 14,
  "model_loaded": true,
  "model_name": "DeepHealthRiskNet",
  "status": "healthy",
  "timestamp": "2026-10-03T07:48:49.437256+00:00"
}
```

Figure 7-Health endpoint response showing model availability

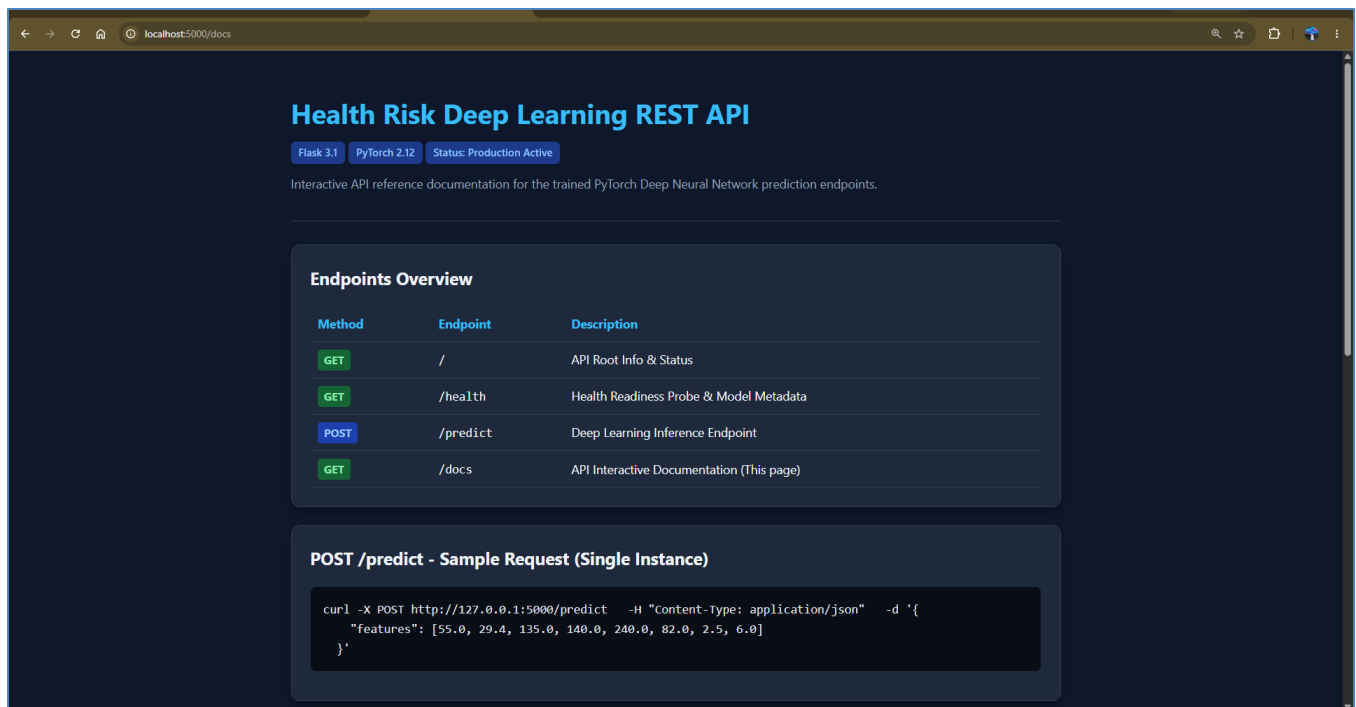


Figure 8-Interactive API documentation at /docs

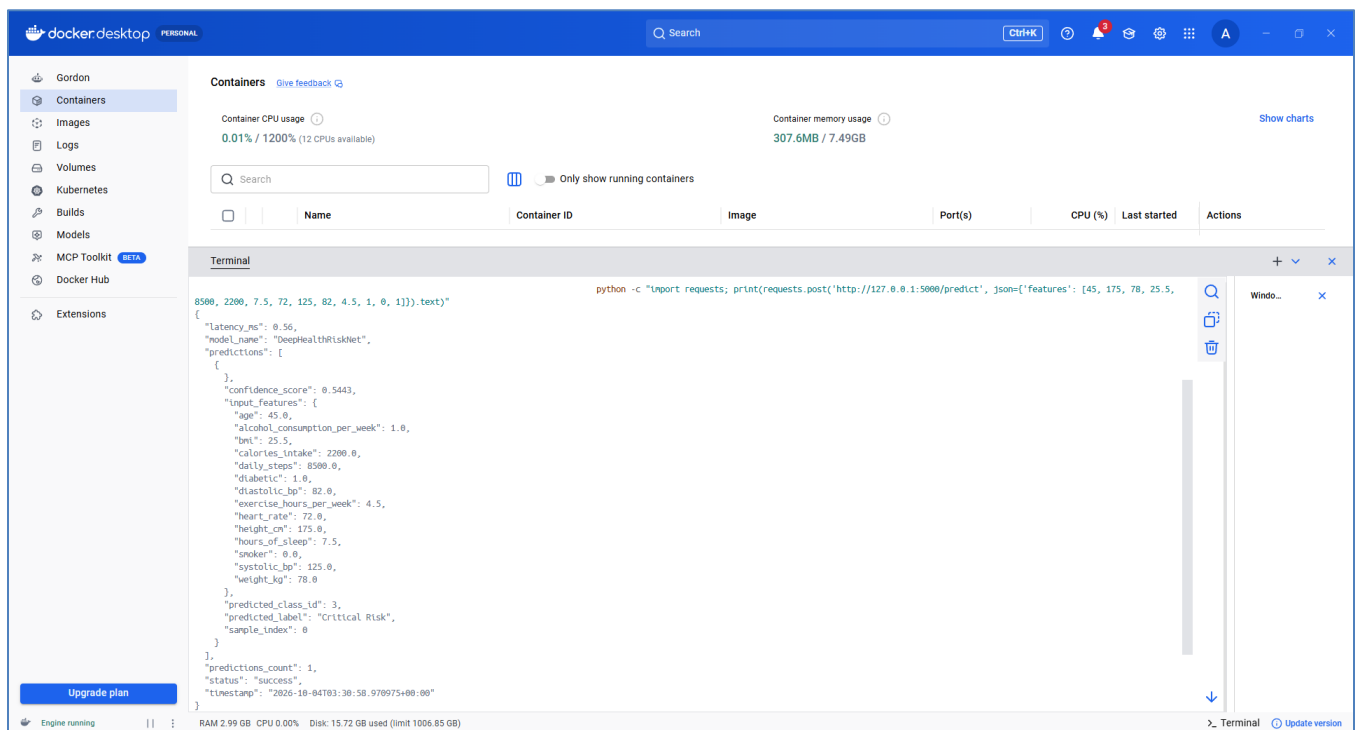


Figure 9-Successful prediction response from /predict

Gordon

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

MCP Toolkit

Docker Hub

Extensions

Containers

Give feedback

Container CPU usage

No containers are running.

Container memory usage

No containers are running.

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	health-risk-api	0cdec3511070	health-risk-flask-api:1.0	5000:5000	N/A	5 minutes ago	

Showing 1 item

Terminal

PS E:\DL_deploy\Int_Deep_Learning\Task_10-Containerizing_Flask_Deep_Learning_API> docker stop health-risk-api

health-risk-api

PS E:\DL_deploy\Int_Deep_Learning\Task_10-Containerizing_Flask_Deep_Learning_API>

Windo...

Upgrade plan

Engine running

RAM 2.75 GB CPU 0.00% Disk: 15.72 GB used (limit 1006.85 GB)

Terminal

Update version

Gordon

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

MCP Toolkit

Docker Hub

Extensions

Containers

Give feedback

Container CPU usage

0.01% / 1200% (12 CPUs available)

Container memory usage

307.6MB / 7.49GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	health-risk-api	0cdec3511070	health-risk-flask-api:1.0	5000:5000	0.01%	0 seconds ago	

Showing 1 item

Terminal

PS E:\DL_deploy\Int_Deep_Learning\Task_10-Containerizing_Flask_Deep_Learning_API> docker stop health-risk-api

health-risk-api

PS E:\DL_deploy\Int_Deep_Learning\Task_10-Containerizing_Flask_Deep_Learning_API> docker start health-risk-api

health-risk-api

PS E:\DL_deploy\Int_Deep_Learning\Task_10-Containerizing_Flask_Deep_Learning_API>

Windo...

Upgrade plan

Engine running

RAM 2.74 GB CPU 0.00% Disk: 15.72 GB used (limit 1006.85 GB)

Terminal

Update version

Page 10

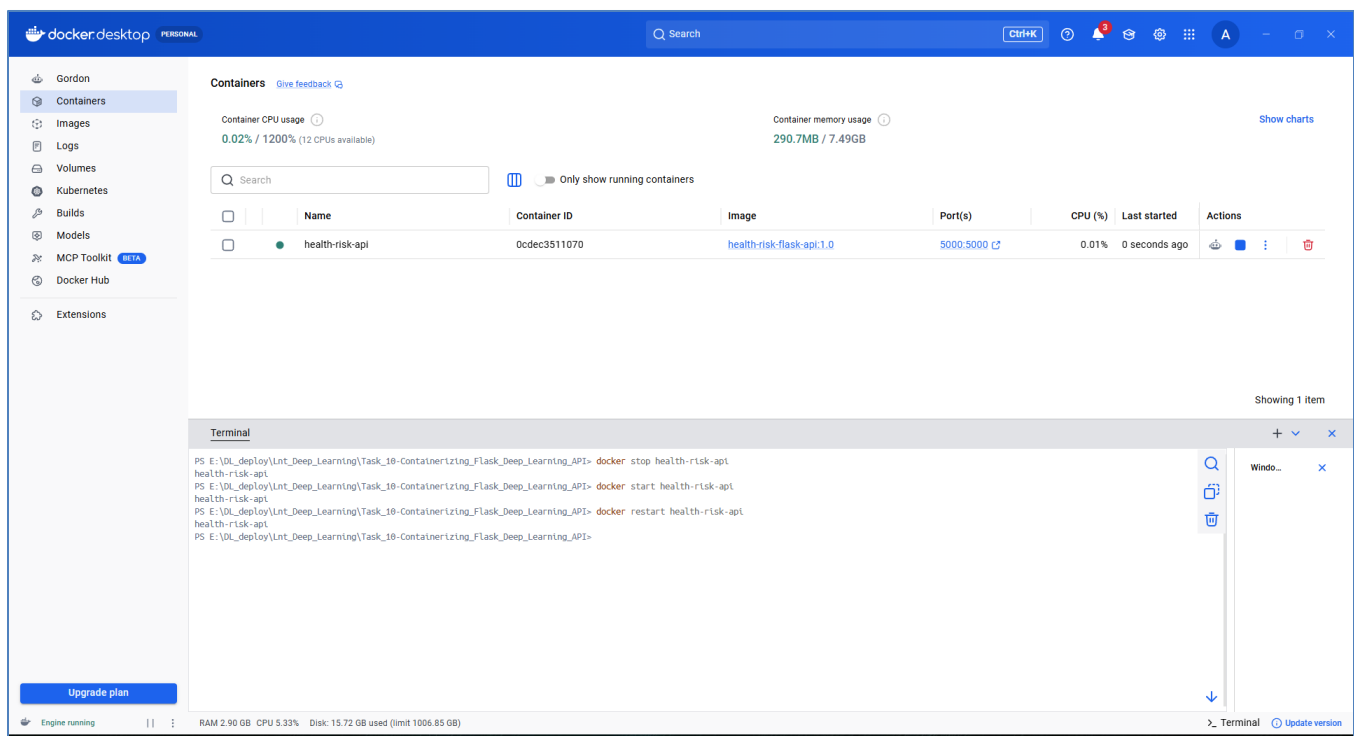
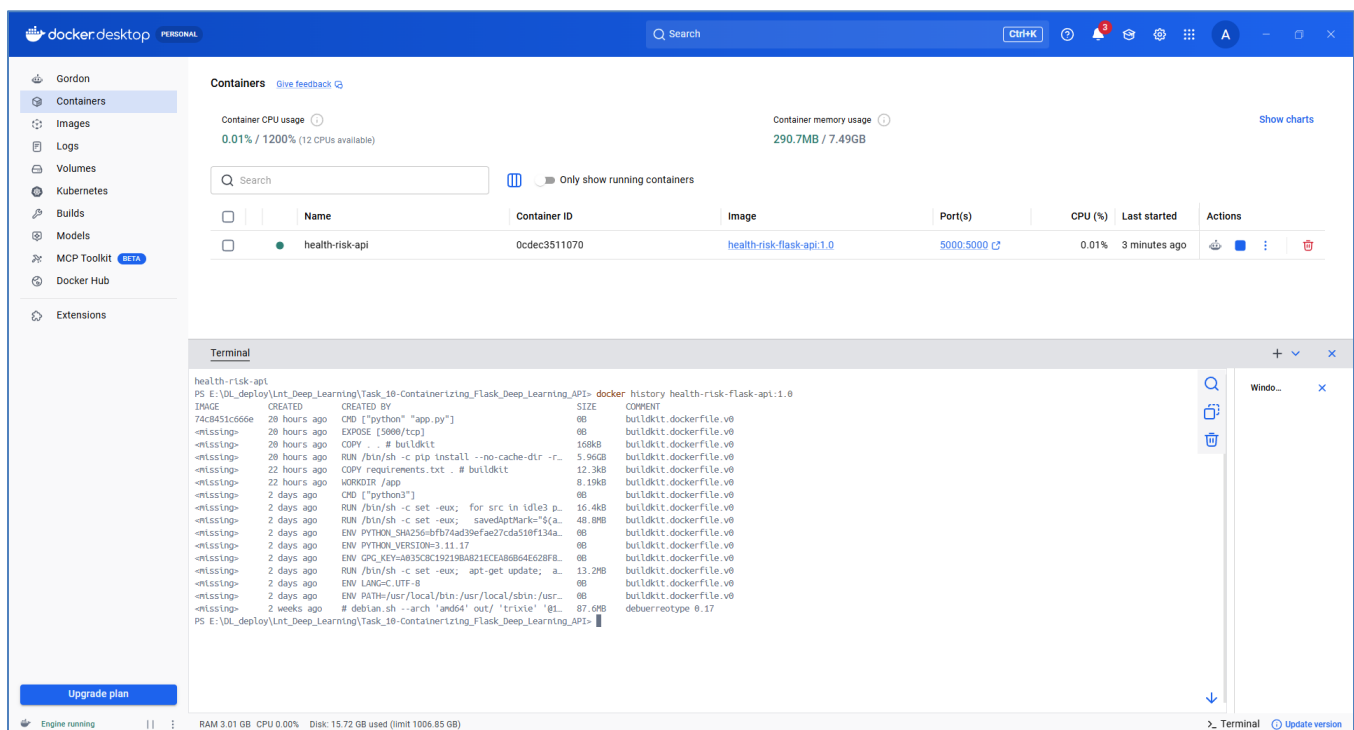


Figure 10-Container lifecycle commands



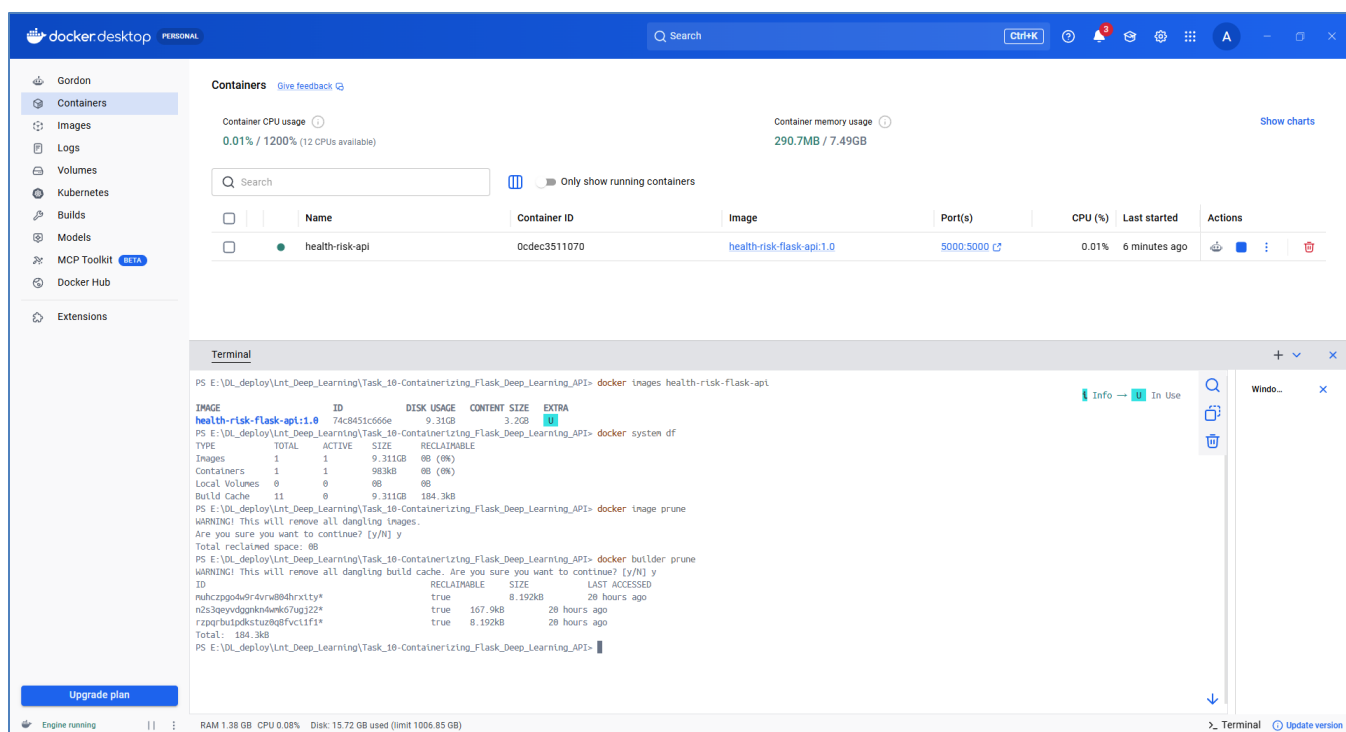


Figure 11-Docker image history / optimization evidence

13. Results

The Flask deep learning API was successfully containerized using Docker. The Docker image was built from the project Dockerfile and a container was launched with host port 5000 mapped to container port 5000. The API endpoints were accessible locally, the health endpoint was used to verify model readiness, and the prediction endpoint was used to verify model inference.

14. Observations

- Docker provided an isolated runtime environment containing the Python dependencies required by the Flask API.
- The trained model and API application could be packaged together for reproducible local deployment.
- Port mapping allowed the API running inside the container to be accessed from the host browser.
- Docker logs provided a convenient method for checking application startup and runtime behavior.
- The slim base image, no-cache package installation, and .dockerignore configuration reduced unnecessary image contents.

15. Conclusion

Task 10 was completed by containerizing and locally deploying the Flask-based deep learning API with Docker. The implementation demonstrated Dockerfile creation, image building, container execution, API and model verification, container lifecycle management, logging, and basic Docker image optimization. The resulting container provides a portable environment for running the Flask deep learning service.

16. Commands Used — Quick Reference

```
docker --version
docker info
docker build -t health-risk-flask-api:1.0 .
docker images
docker run -d --name health-risk-api -p 5000:5000 health-risk-flask-api:1.0
docker ps
docker ps -a
docker logs health-risk-api
docker logs -f health-risk-api
docker stop health-risk-api
docker start health-risk-api
docker restart health-risk-api
docker stats health-risk-api
docker history health-risk-flask-api:1.0
docker inspect health-risk-flask-api:1.0
```

Github:

https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task_10-Containerizing_Flask_Deep_Learning_API

DockerHub:

<https://hub.docker.com/r/adityabahira/health-risk-flask-api>